

Taylex Displays — Lead Engine Architecture

Recommendation & Architecture Review

Prepared by Bibot Core — May 2026 Confidential

Purpose of this document

Taylex Displays runs an automated lead-acquisition engine that sources exhibiting companies from trade-show exhibitor lists, enriches them with decision-maker contacts, and loads them into GoHighLevel (GHL) for outreach.

This document does three things:

1. Confirms **how the lead-acquisition pipeline should actually work** (corrected for how exhibitor data is really published)
 2. Explains the **limits of the current n8n-based approach** at scale
 3. Sets out the **advantages of building the engine as a Bibot design on Vercel + Apify + Supabase**, and the architecture we recommend
-

1. The lead-acquisition pipeline — how it really works

Taylex's customers are **the exhibitors themselves** — companies that book a stand at a trade show. By definition, the exhibitor list of any relevant show is a list of companies that need a stand built. That makes the targeting clean:

Relevant show → its exhibitor list → every company on it is a prospect.

The two-hop sourcing reality

A common misconception is that show aggregators (10times, eventseye, ExpoDatabase) publish the exhibitor companies. **They do not.** They are show *calendars*. The actual exhibitor names live on each individual show's own website.

```
Show aggregator
  (10times / eventseye / ExpoDatabase)
    |   gives us: show name, dates, sector, location, OFFICIAL SHOW URL
    ▼
Individual show website
  (e.g. anuga.com → "Exhibitor Directory")
    |   gives us: the actual company names + websites + stand numbers
    ▼
Company list (the prospects)
```

Where the company names actually are

On each show's own site, the data lives on a page called **"Exhibitor List" / "Exhibitor Directory" / "Who's Exhibiting" / "Floor Plan" / "Exhibitor Search."**

The software vendors behind these directories (Map Your Show, Swapcard, ExpoFP, ExpoPlatform, a2z) do **not** publish a master list — they power the directory *inside* each show's website. You always enter through the show's own URL.

Three practical rules for reliable scraping

1. **Scrape the JSON, not the HTML.** Exhibitor directories almost always load their data from a background JSON API. Hitting that endpoint directly is far more robust than parsing rendered HTML — no pagination or anti-bot fragility.
2. **Build one extractor per platform, not per show.** Roughly 70% of shows run on a handful of directory platforms. Platform-level extractors are reusable across hundreds of shows.
3. **Scrape close to the event, and re-scrape.** Exhibitor lists fill up in the weeks before a show and grow as companies register late. An early scrape misses companies.

Capture the company domain during scraping (key insight)

Exhibitor directory entries almost always include the company's website link. **Capture it.** A verified domain is what makes downstream enrichment reliable — a company name alone is ambiguous.

Capture: **name + domain + stand number + show.**

The enrichment step

Since **BetterContact performs company → decision-maker discovery** (not just email verification of known people), the flow is clean and needs no separate people-database step:

```
Exhibitor name + domain
  ▼
BetterContact (waterfall: finds the decision maker + verified email)
  ▼
GHL contact (tagged with show, sector, decision-maker role)
```

A filter step worth adding

Not every exhibitor is a prospect. Exhibitor lists include other stand builders (direct competitors), companies too small to have a stand budget, and firms that build in-house. A cheap exclusion pass (competitor keywords + company size) keeps the list clean and protects email deliverability — far better than loading tens of thousands of raw names into GHL.

2. The limits of n8n at scale

n8n is an excellent tool for quickly connecting APIs and is the right choice for a fast MVP. But as the backbone of a scalable lead-generation platform, it runs into structural limits:

Limit	Why it matters for Taylex
Heavy datasets in memory	n8n holds workflow data in memory between nodes. Processing tens of thousands of exhibitor rows per run becomes slow and unstable.
Long-running jobs don't fit	Scraping a large directory can take minutes to hours. n8n's execution model isn't built for long jobs — timeouts and partial failures.
Weak state across runs	n8n has no real database of its own. To track "which shows have we scraped," "which companies are enriched," you bolt on an external DB anyway — so you're already half-way to a code solution.
Hard to test & version-control	Workflows are JSON blobs. There's no meaningful code review, no automated tests, no clean git history. Debugging complex multi-branch logic in the visual editor gets painful fast.
Doesn't templatzize for multiple clients	Running the same engine for a second or third client means cloning and hand-editing workflows. There's no clean "deploy the same engine for a new client" path.
Per-execution cost & vendor lock-in	n8n cloud bills per execution and the whole system lives at a single hosted URL — a single point of dependency.

In short: n8n is great glue for a prototype. It is not a robust product backbone once volume, repeatability, and maintainability matter.

3. The advantages of building it with Bibot + Vercel

Building the engine the way we built **Apulia Power** — Vercel functions + Apify + Supabase, presented through a Bibot design — gives a system that actually scales.

Advantage	What it means in practice
Purpose-built scraping	Apify is built for scraping at scale — proxies, headless browsers, retries, parallelism. It handles the heavy lifting far better than n8n. (Apify is the right tool <i>either way</i> — the question is only what orchestrates it.)
Real state & job queue	Supabase holds the shows, exhibitors, and enrichment queue as proper tables — the single source of truth, queryable and reliable.
Testable, version-controlled code	The orchestration lives in real code: git history, code review, automated tests, safe rollbacks.
Templatizable for multiple clients	The same engine deploys for a new client by configuration, not by cloning workflows. This is the foundation of a reusable Bibot product.
Lower running cost at volume	No per-execution workflow billing — just Vercel + Apify + Supabase.
One integrated experience	The data flows straight into a Bibot command-center design: show pipeline, exhibitor counts, enrichment hit-rate, and the show-ROI view that tells Taylex which shows are worth scraping next. No separate dashboards or digest emails.
Proven pattern	This isn't theoretical — it's the exact architecture already shipped for Apulia Power (cached data synced from GHL, chunked imports, a scheduled worker).

The one constraint to design around

Vercel serverless functions have an execution-time limit (max ~300 seconds on the Pro plan). **A long scrape cannot run inside a Vercel function** — it would time out. This is exactly why Apify exists: Apify runs the long scraping job, then calls back to a short-lived Vercel endpoint with the results. The architecture is designed around this from the start.

4. Recommended architecture

STEP 1 – SOURCE SHOWS

Apify actor scrapes aggregators (10times / eventseye)
filtered by sector + region (UK / Europe / Dubai)

▼ list of shows + official URLs → Supabase: shows

STEP 2 – SCRAPE EXHIBITORS

Apify actors (one per directory platform) scrape each
show's exhibitor directory via its JSON API

captures: name + domain + stand no. + show
▼ webhook on completion

Vercel endpoint /api/taylex/scrape-webhook (short-lived)

▼ → Supabase: exhibitors (deduped, competitor-filtered)

STEP 3 – ENRICH

Supabase pg_cron worker pulls from enrichment_queue

▼ BetterContact (company + domain → decision maker)
▼ → Supabase: exhibitors (enriched)

STEP 4 – LOAD TO CRM

Push enriched contacts to GHL

tagged: show · sector · decision-maker role · score
▼ smart-list segmentation

STEP 5 – COMMAND CENTER (Bibot design)

Show pipeline · exhibitor counts per show ·
enrichment hit-rate · outreach funnel · SHOW ROI

Supabase schema (core tables)

taylex_shows

id · name · sector · region · show_url · directory_platform ·
starts_at · scraped_at · exhibitor_count · status

taylex_exhibitors

id · show_id · company_name · domain · stand_number ·
excluded (bool) · exclusion_reason · enrichment_status ·
decision_maker_name · decision_maker_role · email · phone ·
ghl_contact_id · created_at

taylex_enrichment_queue

id · exhibitor_id · status · attempts · last_attempt_at · provider_response

What stays where

Layer	Tool	Why
Heavy scraping	Apify	Purpose-built; the right tool regardless of architecture
Orchestration & APIs	Vercel functions	Short-lived, testable, version-controlled
State & job queue	Supabase	Single source of truth; pg_cron worker for scheduling
Enrichment	BetterContact	Company → decision-maker + verified email
CRM & outreach	GHL	Where Taylex already runs sequences
Presentation & intelligence	Bibot design	Show pipeline, hit-rate, and show-ROI command center

5. The Bibot Command Center (the highest-value deliverable)

Today Taylex processes shows **blindly** — there's nothing that tells them which shows actually produce leads that convert. The single most valuable thing Bibot adds is a **Show ROI view**:

- **Show pipeline** — which shows are queued, scraped, enriched, loaded
- **Exhibitor counts per show** — how many prospects each show yielded
- **Enrichment hit-rate** — % of companies where BetterContact found a decision maker
- **Outreach funnel** — sent → open → reply → booked, per show / domain / sequence
- **Show ROI** — which shows produce the best leads and conversions, so scraping effort goes where it pays

This turns "scrape every show we can find" into "scrape the shows that actually convert."

6. Recommendation summary

Question	Answer
Is the lead-acquisition logic correct?	Yes — exhibitor lists are the right source; targeting the exhibitors (not organisers) is correct
Where do the company names come from?	Each individual show's exhibitor directory (via its JSON API), not the aggregators
Does BetterContact close the gap?	Yes — it does company → decision-maker discovery, so no Apollo middle-step needed
Is n8n the right backbone for scale?	No — fine for an MVP, but it doesn't scale, test, or templatize as a product
What should we build instead?	The Apulia Power pattern : Apify + Vercel + Supabase, presented through a Bibot command-center design

The decision that drives everything

Is this only for Taylex, or is Bibot building a reusable lead engine to run for multiple clients?

- **Only Taylex, and the live n8n system works** → keep Apify + n8n for the engine, and have Bibot own the command-center dashboard on top.
- **A scalable, multi-client platform** → build the engine as code (Vercel + Apify + Supabase) from the start. This is more robust, testable, and templatizable — and it's a pattern we have already shipped.

Our recommendation is the **code-based platform** if there is any ambition to scale beyond a single client.

Bibot Core — Taylex Displays Lead Engine Architecture — May 2026 — Confidential